

GUIA.md - Guía de Funciones Consolidadas



Resumen Conceptual: Teoría de la Información y la Comunicación

Este proyecto implementa los conceptos fundamentales de **codificación eficiente**, **modelado de canales ruidosos** y **límites teóricos de transmisión**, conforme a las Unidades 4, 5 y 6 del curso.

IMPORTANTE: Este archivo standalone contiene ÚNICAMENTE las funciones utilizadas en los trabajos prácticos tp4, tp5 y tp6.



Índice

- [UNIDAD 4: Compresión y Errores](#)
 - [UNIDAD 5: El Canal y sus Propiedades](#)
 - [UNIDAD 6: Capacidad y Probabilidad de Error](#)
 - [Catálogo de Funciones](#)
-

UNIDAD 4: Compresión y Errores

Fundamentos Teóricos de la Compresión

- **Primer Teorema de Shannon (Codificación sin Ruido):** Establece la cota mínima para la longitud media (L) de un código instantáneo r -ario. La longitud media siempre debe ser mayor o igual a la entropía de la fuente: $L \geq H_r(S)$
- **Cota de Compresión:** Para la extensión de orden n , la longitud media por símbolo L_n/n está acotada por:

$$H_r(S) \leq L_n/n < H_r(S) + 1/n$$

- **Rendimiento y Redundancia:**
 - Rendimiento o Eficiencia: $\eta = H_r(S) / L$
 - Redundancia: $1 - \eta$

Algoritmos de Codificación

- **Códigos de Huffman:** Algoritmo óptimo que produce códigos instantáneos y compactos
- **Códigos de Shannon-Fano:** Procedimiento subóptimo, cumple $L \leq H(S) + 2$
- **Run Length Coding (RLC):** Codifica secuencias idénticas como pares (símbolo, repeticiones)

Control de Errores

- **Distancia de Hamming (d):** Número de bits que difieren entre dos palabras de código
 - Detección: hasta $d - 1$ errores
 - Corrección: hasta $\lfloor (d-1)/2 \rfloor$ errores
 - **Paridad Cruzada (VRC/LRC):** Distancia de Hamming = 4, detecta hasta 3 errores y corrige 1 error simple
-

UNIDAD 5: El Canal y sus Propiedades

Caracterización del Canal

- **Canal de Información:** Definido por:
 - Alfabeto de entrada (A)
 - Alfabeto de salida (B)
 - Matriz de probabilidades condicionales $P(b_j|a_i)$

Métricas de Entropía y Ruido

- **Entropía a Priori $H(A)$:** Incertidumbre promedio de la entrada
- **Entropía de la Salida $H(B)$:** Incertidumbre promedio de la salida
- **Entropía Afín $H(A,B)$:** Incertidumbre del evento simultáneo (a_i, b_j)
- **Equivocación (Ruido) $H(A|B)$:** Pérdida de información causada por el canal
- **Pérdida $H(B|A)$:** Incertidumbre de la salida conocida la entrada

Información Mutua

- **Definición:** $I(A,B) = H(A) - H(A|B)$
- **Simetría:** $I(A,B) = I(B,A) = H(B) - H(B|A)$

UNIDAD 6: Capacidad y Probabilidad de Error

Capacidad del Canal (C)

- **Definición:** Máximo valor de Información Mutua

$$C = \max_{P(a_i)} \{ I(A,B) \}$$

- **Casos Especiales:**
 - Canal Sin Ruido: **$C = \log(\text{nro de entradas})$**
 - Canal Determinante: **$C = \log(\text{nro de salidas})$**
 - Canal Uniforme: Fórmula específica según distribución

Canales Compuestos y Reducción

- **Canales en Serie:** Matriz compuesta = multiplicación de matrices individuales
- **Reducción Suficiente:** Permite combinar salidas sin perder información mutua

Catálogo de Funciones

Cómo Importar

```
# Importar el módulo completo
from parcial2_template.total import utils_standalone as utils

# Usar funciones
entropia = utils.calculateH(probabilidades)

# O importar funciones específicas
from parcial2_template.total.utils_standalone import calculateH, huffman
```

 Mapeo Teoría → Implementación

UNIDAD 4: Compresión (Funciones usadas en tp4/tp5/tp6)

Concepto Teórico	Función	Importar con
Entropía $H(S)$	<code>calculateH(P)</code>	<code>from parcial2_template.total.utils_standalone import calculateH</code>
Entropía base- r $H_r(S)$	<code>calculateHr(pbs, r)</code>	<code>from parcial2_template.total.utils_standalone import calculateHr</code>
Información $I(pi)$	<code>calculateI(pi)</code>	<code>from parcial2_template.total.utils_standalone import calculateI</code>
Información base- r $I_r(pi)$	<code>calculateIr(p, r)</code>	<code>from parcial2_template.total.utils_standalone import calculateIr</code>
Longitud Media L	<code>getLengthMedCodigo(cods, pbs)</code>	<code>from parcial2_template.total.utils_standalone import getLengthMedCodigo</code>
Alfabeto del código	<code>getAlfabetoCodigo(cods)</code>	<code>from parcial2_template.total.utils_standalone import getAlfabetoCodigo</code>
Longitudes	<code>getLengthsCodigo(cods)</code>	<code>from parcial2_template.total.utils_standalone import getLengthsCodigo</code>
Rendimiento η	<code>rendimientoCodigo(C, P)</code>	<code>from parcial2_template.total.utils_standalone import rendimientoCodigo</code>
Redundancia	<code>redundanciaCodigo(C, P)</code>	<code>from parcial2_template.total.utils_standalone import redundanciaCodigo</code>
Primer Teorema Shannon	<code>teoremaShannon(C, P, n)</code>	<code>from parcial2_template.total.utils_standalone import teoremaShannon</code>
Teorema Shannon (extendido)	<code>teoremaShannonExtending(C, P, n)</code>	<code>from parcial2_template.total.utils_standalone import teoremaShannonExtending</code>

Algoritmos de Codificación (UNIDAD 4)

Algoritmo	Función	Importar con
-----------	---------	--------------

Algoritmo	Función	Importar con
Huffman	<code>huffman(P)</code>	<code>from parcial2_template.total.utils_standalone import huffman</code>
Shannon-Fano	<code>shannonfano(P)</code>	<code>from parcial2_template.total.utils_standalone import shannonfano</code>
RLC (Run Length Coding)	<code>rlc(message)</code>	<code>from parcial2_template.total.utils_standalone import rlc</code>
Codificar con Huffman/SF	<code>codificar(message, alf, C)</code>	<code>from parcial2_template.total.utils_standalone import codificar</code>
Codificar con diccionario	<code>codificar_dict(message, C)</code>	<code>from parcial2_template.total.utils_standalone import codificar_dict</code>
Decodificar	<code>decodificar(data, C, alf)</code>	<code>from parcial2_template.total.utils_standalone import decodificar</code>
Decodificar con diccionario	<code>decodificar_dict(data, C)</code>	<code>from parcial2_template.total.utils_standalone import decodificar_dict</code>

Control de Errores (UNIDAD 4)

Concepto	Función	Importar con
Distancia de Hamming d	<code>hamming(C)</code>	<code>from parcial2_template.total.utils_standalone import hamming</code>
Errores detectables (d-1)	<code>erroresDetectables(C)</code>	<code>from parcial2_template.total.utils_standalone import erroresDetectables</code>
Errores corregibles [(d-1)/2]	<code>erroresCorregibles(C)</code>	<code>from parcial2_template.total.utils_standalone import erroresCorregibles</code>
Codificar multiparidad	<code>encodeMultiparidad(message, par)</code>	<code>from parcial2_template.total.utils_standalone import encodeMultiparidad</code>
Decodificar multiparidad	<code>decodeMultiparidad(byte_array, par)</code>	<code>from parcial2_template.total.utils_standalone import decodeMultiparidad</code>

Extensiones de Fuentes (UNIDAD 4)

Concepto	Función	Importar con
----------	---------	--------------

Concepto	Función	Importar con
Generar extensión orden n	<code>generateExtensionsFromLL(alf, prob, n)</code>	<code>from parcial2_template.total.utils_standalone import generateExtensionsFromLL</code>
Extensión desde dict	<code>generateExtensionsFromD(S, n)</code>	<code>from parcial2_template.total.utils_standalone import generateExtensionsFromD</code>
Extensión de probabilidades	<code>generateExtensionsP(prob, n)</code>	<code>from parcial2_template.total.utils_standalone import generateExtensionsP</code>
Construir fuente	<code>buildS(source)</code>	<code>from parcial2_template.total.utils_standalone import buildS</code>
Obtener ocurrencias	<code>getSymbolOcurrences(phrase)</code>	<code>from parcial2_template.total.utils_standalone import getSymbolOcurrences</code>

Almacenamiento (UNIDAD 4)

Concepto	Función	Importar con
Guardar comprimido	<code>saveComprimido(bits, filename, path)</code>	<code>from parcial2_template.total.utils_standalone import saveComprimido</code>
Recuperar comprimido	<code>recoverComprimido(filename, path)</code>	<code>from parcial2_template.total.utils_standalone import recoverComprimido</code>
Tasa de compresión	<code>tasaCompresion(message, compressed)</code>	<code>from parcial2_template.total.utils_standalone import tasaCompresion</code>

UNIDAD 5: Canales - Probabilidades

Concepto	Función	Importar con
P(ai) - Prob. a priori	<code>getProbabilidadPriori(message)</code>	<code>from parcial2_template.total.utils_standalone import getProbabilidadPriori</code>
P(B A) - Matriz del canal	<code>getPrioriMatrixFull(fnt, cds, _input, _output)</code>	<code>from parcial2_template.total.utils_standalone import getPrioriMatrixFull</code>
P(bj) - Prob. de salida	<code>getProbsOutSymbols(Pinitial, channel)</code>	<code>from parcial2_template.total.utils_standalone import getProbsOutSymbols</code>
P(A B) - Prob. a posteriori	<code>getPosterioriMatrix(Pinitial, channel)</code>	<code>from parcial2_template.total.utils_standalone import getPosterioriMatrix</code>

Concepto	Función	Importar con
P(ai, bj) - Evento simultáneo	<code>getMatrixSimultaneusEvent(Pinitial, channel)</code>	<code>from parcial2_template.total.utils_standalone import getMatrixSimultaneusEvent</code>

UNIDAD 5: Canales - Entropías

Concepto	Función	Importar con
H(A) - Entropía a priori	<code>calculateHPriori(Pa)</code>	<code>from parcial2_template.total.utils_standalone import calculateHPriori</code>
H(A bj) - Entropía a posteriori	<code>calculateHPosteriori(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import calculateHPosteriori</code>
H(B) - Entropía de salida	<code>calculateHPosterioriTotal(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import calculateHPosterioriTotal</code>
H(A B) - Equivocación/Ruido	<code>calculateRuido(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import calculateRuido</code>
H(B A) - Pérdida	<code>calculatePerdida(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import calculatePerdida</code>
H(A,B) - Entropía afín	<code>calculateHCanal(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import calculateHCanal</code>

UNIDAD 5: Información Mutua

Concepto	Función	Importar con
I(A,B) fórmula simple	<code>informacionMutuaABSimple(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import informacionMutuaABSimple</code>
I(B,A) fórmula simple	<code>informacionMutuaBASimple(Pa, channel)</code>	<code>from parcial2_template.total.utils_standalone import informacionMutuaBASimple</code>

UNIDAD 5: Propiedades de Canales

Propiedad	Función	Importar con
Canal sin ruido	<code>isCanalNoRuido(channel)</code>	<code>from parcial2_template.total.utils_standalone import isCanalNoRuido</code>

Propiedad	Función	Importar con
Canal determinante	<code>isCanalDeterminante(channel)</code>	<pre>from parcial2_template.total.utils_standalone import isCanalDeterminante</pre>

UNIDAD 6: Capacidad del Canal

Concepto	Función	Importar con
C (general)	<code>calcularCapacidad(channel)</code>	<pre>from parcial2_template.total.utils_standalone import calcularCapacidad</pre>
C (binario por búsqueda)	<code>calculateCapacidadBinario(channel, step)</code>	<pre>from parcial2_template.total.utils_standalone import calculateCapacidadBinario</pre>

UNIDAD 6: Probabilidad de Error

Concepto	Función	Importar con
PE (regla ML)	<code>probabilidadError(channel, P)</code>	<pre>from parcial2_template.total.utils_standalone import probabilidadError</pre>

UNIDAD 6: Canales Compuestos

Concepto	Función	Importar con
Canales en serie	<code>generarComposedChannel(channelA, channelB)</code>	<pre>from parcial2_template.total.utils_standalone import generarComposedChannel</pre>
Reducción suficiente	<code>isReduccionSuficiente(channel, col1, col2)</code>	<pre>from parcial2_template.total.utils_standalone import isReduccionSuficiente</pre>
Obtener canal reducido	<code>getReducedChannel(channel, squareLimit)</code>	<pre>from parcial2_template.total.utils_standalone import getReducedChannel</pre>
Combinar columnas	<code>combinateCols(channel, col1, col2)</code>	<pre>from parcial2_template.total.utils_standalone import combineCols</pre>
Canal determinante	<code>getCanalDeterminante(channel, col1, col2)</code>	<pre>from parcial2_template.total.utils_standalone import getCanalDeterminante</pre>

Utilidades de Matrices

Función	Importar con
<code>printMatrix(matrix)</code>	<pre>from parcial2_template.total.utils_standalone import printMatrix</pre>

Función	Importar con
<code>printMatrixVerbose(matrix, row_labels, col_labels)</code>	<code>from parcial2_template.total.utils_standalone import printMatrixVerbose</code>

Ejemplos de Uso

Ejemplo 1: Calcular Entropía y Rendimiento (UNIDAD 4)

```
from parcial2_template.total.utils_standalone import calculateH, huffman,
rendimientoCodigo

# Probabilidades de la fuente
P = [0.5, 0.3, 0.2]

# Calcular entropía
H = calculateH(P)
print(f"Entropía: {H:.4f} bits")

# Generar código de Huffman
C = huffman(P)
print(f"Código Huffman: {C}")

# Calcular rendimiento
eta = rendimientoCodigo(C, P)
print(f"Rendimiento: {eta*100:.2f}%")
```

Ejemplo 2: Análisis de Canal (UNIDAD 5)

```
from parcial2_template.total.utils_standalone import (
    calculateRuido, calculatePerdida, informacionMutuaABSimple
)

# Canal binario simétrico con p=0.1
canal = [
    [0.9, 0.1],
    [0.1, 0.9]
]

# Probabilidades a priori uniformes
Pa = [0.5, 0.5]

# Métricas del canal
ruido = calculateRuido(Pa, canal)
perdida = calculatePerdida(Pa, canal)
info_mutua = informacionMutuaABSimple(Pa, canal)

print(f"Ruido H(A|B): {ruido:.4f}")
print(f"Pérdida H(B|A): {perdida:.4f}")
print(f"I(A,B): {info_mutua:.4f}")
```

Ejemplo 3: Control de Errores (UNIDAD 4)

```
from parcial2_template.total.utils_standalone import (
    hamming, erroresDetectables, erroresCorregibles
)

# Código de ejemplo
codigo = ['000', '011', '101', '110']

# Calcular distancia de Hamming
d = hamming(codigo)
print(f"Distancia de Hamming: {d}")

# Capacidades
detectables = erroresDetectables(codigo)
corregibles = erroresCorregibles(codigo)

print(f"Errores detectables: {detectables}")
print(f"Errores corregibles: {corregibles}")
```

Ejemplo 4: Capacidad de Canal Binario y Probabilidad de Error (UNIDAD 6)

```
from parcial2_template.total.utils_standalone import (
    calculateCapacidadBinario, probabilidadError
)

# Canal binario simétrico
canal = [
    [0.9, 0.1],
    [0.1, 0.9]
]

# Calcular capacidad (búsqueda numérica)
p_opt, C = calculateCapacidadBinario(canal, step=0.01)
print(f"P(a1) óptima: {p_opt:.4f}")
print(f"Capacidad: {C:.4f} bits")

# Calcular probabilidad de error con regla ML
Pa = [0.5, 0.5]
PE = probabilidadError(canal, Pa)
print(f"Probabilidad de Error (PE): {PE:.4f}")
```



Notas Importantes

1. **Solo funciones usadas:** Este standalone contiene ÚNICAMENTE las funciones utilizadas en tp4, tp5 y tp6
2. **Sin dependencias internas:** El archivo es completamente standalone
3. **Imports necesarios:** Solo usa librerías estándar de Python: `math`, `random`, `os`
4. **Compatibilidad:** Compatible con Python 3.6+



Lista Completa de Funciones Incluidas

Las siguientes funciones están disponibles en `utils_standalone.py`:

Compresión y Códigos:

- `calculateH, calculateI, calculateHr, calculateIr`
- `huffman, shannonfano, rlc`
- `getAlfabetoCodigo, getLengthsCodigo, getLengthMedCodigo`
- `rendimientoCodigo, redundanciaCodigo`
- `teoremaShannon, teoremaShannonExtending`
- `codificar, decodificar, codificar_dict, decodificar_dict`
- `saveComprimido, recoverComprimido, tasaCompresion`

Errores:

- `hamming, erroresDetectables, erroresCorregibles`
- `encodeMultiparidad, decodeMultiparidad`

Extensiones:

- `generateExtensionsFromLL, generateExtensionsFromD, generateExtensionsP`
- `buildS, getSymbolOcurrences`

Canales:

- `getProbabilidadPriori, getPrioriMatrixFull`
- `getProbsOutSymbols, getPosterioriMatrix, getMatrixSimultaneusEvent`
- `calculateHPriori, calculateHPosteriori, calculateHPosterioriTotal`
- `calculateRuido, calculatePerdida, calculateHCanal`
- `informacionMutuaABSimple, informacionMutuaBASimple`
- `isCanalNoRuido, isCanalDeterminante`

Capacidad y Probabilidad de Error (Unidad 6):

- `calcularCapacidad, calculateCapacidadBinario`
- `probabilidadError`

Canales en Serie:

- `generarComposedChannel, isReduccionSuficiente, getReducedChannel`
- `combinareCols, getCanalDeterminante`

Utilidades:

- `printMatrix, printMatrixVerbose`

Total de funciones: 80 (incluyendo dependencias internas)

Archivo: `parcial2_template/total/utils_standalone.py`

Líneas: ~1050